

SQLite ile Quiz Veri Tabanı

SQL / Veri Tabanı Dersi – Haftalık Proje Raporu

Hazırlayan: Yağız Efe Çiçekçisoğlu · 124200053 | Son teslim: 7 Ekim 2026

İncelemeden önce bilinmesi gerekenler

- İki adres:** Hoca ve öğrenciyi aynı anda denemek için birini `http://127.0.0.1:5000`, diğerini `http://localhost:5000` adresinde açın. Aynı adresteki iki sekme birbirinin oturumunu kapatır. (Alternatif: farklı tarayıcı ya da gizli pencere.)
- Açılıştaki “örnek veri bayat” mesajı:** Örnek veride bir quizin (w02-s01) “sürüyor” durumunda olmasını istedim, çünkü ödevin istediği başlamamış, süren ve bitmiş durumların üçü de veride hazır görünsün ve canlı takip ilk açılışta dolu olsun istedim. Ancak `seed.py` bu quizi çalıştırdığı andan 30 sn önce başlamış olarak kaydediyor ve proje günler sonra açıldığında quiz “günlerdir sürüyor” (ör. +250000 sn uzatma) görünecekti. Bunu önlemek için, hocam projeyi açtığı anda verinin güncel olması amacıyla **5 dakikayı sınır alan bir kontrol ekledim:** `python app.py` açılırken süren quiz 5 dakikadan uzun süredir açıksa veriyi `seed.py` ile yeniden kuruyor ve terminale bunu yazıyor. Böylece quiz 0:30 'dan başlıyor. Seed her seferinde aynı veriyi ürettiği için (`random.seed(42)`) silinen önemli bir şey olmuyor, değişen tek şey süren quizin başlangıç saati. Sınırı 3 dakika yapmadım, çünkü quizin resmî süresi 3 dakika ve gerçek uzatmaları (+10, +30 sn) da bayat sayardı. Kontrol yalnızca açılışta yapılıyor, uygulama çalışırken veriye dokunulmuyor.
- Sayaç:** Süren quiz 0:30 civarından başlar. 3 dakika dolunca uzatmayı gösterir (3:12 (+12 sn)). Quiz, hoca **Bitir** diyene kadar sürer.
- Aynı anda tek quiz:** w02-s02'yi başlatmak için önce w02-s01'i **Bitir**'in.
- Sarı kutular hata değildir:** Veritabanının reddettiği işlemler sarı “veri bütünlüğü kuralı” kutusunda açıklanır.
- Live:** Sayfaların kendiliğinden yenilenmesi beklenen bir davranıştır. Başlat ve Bitir öğrenci ekranına 1 sn içinde yansır.
- Demo öğrencisi:** 20240023@ogrenci.edu.tr / ogrenci123 . Süren quiz'e katılmış ama henüz cevap vermemiş.
- DB Browser:** `quiz.db` doğrudan açılırsa açılış kontrolü çalışmaz. Taze süre için önce `python seed.py` çalıştırın.
- Bilinen sınırlamalar:** Windows'ta test edildi. Mac'te `python3` gerekebilir, 5000 portu AirPlay ile çakışabilir.

İlerleme Durumu

Adım	İş	Çıktı	Durum
0	Planlama, gereksinim analizi, ortam kurulumu	Bu rapor, git deposu	Tamamlandı
1	Veri modeli: tablolar ve ilişkiler	ER diyagramı (Bölüm 4)	Tamamlandı
2	Şema: anahtarlar, kısıtlar, indeksler, trigger'lar	schema.sql	Tamamlandı
3	Örnek veri: 50+ kullanıcı, 5 oturum, 100 soru, cevaplar	seed.py , quiz.db	Tamamlandı
4	Sorgular: canlı takip, sonuçlar, sıralama, kayıt sayıları	queries.sql	Tamamlandı
5	Veri bütünlüğü testleri	testler.py	Tamamlandı
6	Dokümantasyon	README.md	Tamamlandı
7	Arayüz (ek puan)	app.py , templates/	Tamamlandı

1. Projenin Amacı ve Hedefleri

Derste kullanılan quiz sisteminin veri katmanını SQLite üzerinde kurmak: kullanıcıların katılımını, soruları, cevapları ve oturumların ilerleyişini tutarlı biçimde saklamak.

Hedef (ödev)	Projedeki karşılığı
En az 50 kullanıcı	Betikle üretilen öğrenci kayıtları + 1 hoca hesabı
5 oturum, en az 100 farklı soru	5 quiz oturumu $\times$ 20 soru = 100 soru
Cevaplar kullanıcı, oturum ve soruya bağlı	cevaplar tablosunda üç yabancı anahtar
Oturum durumları ve zamanları, SQL ile izlenebilir ilerleme	baslamadi $\rightarrow$ suruyor $\rightarrow$ bitti , başlangıç/bitiş zamanları, canlı takip sorgusu

2. Mevcut Sistemin Analizi ve Tasarım Kararları

2.1 Gerçek quiz sisteminin özellikleri

- Quizler haftalara göre gruplanır; her hafta birkaç **Session** (oturum) vardır. Örnek kod: w02-s01 .
- Her quiz 5 adet çoktan seçmeli (A/B/C/D) sorudan oluşur.
- Sonuç puan olarak değil, yüzde olarak gösterilir: $5/5 \cdot \%100$.
- Quiz açıkken sorular arasında geçiş ve cevap değiştirme serbesttir.
- Resmî süre 3 dakikadır; hoca quizi elle başlatıp bitirir. Süre aşılsa sayaç aşımı (+1, +10, +30 sn...) gösterir.
- Her öğrencinin kayıtlı bir e-postası ve şifresi vardır.

2.2 "Oturum" kavramının yorumu

Karar: Bir **oturum** = gerçek sistemdeki tek bir **Session**, yani hocanın başlatıp bitirdiği tek bir quiz.

Gerçek sistemde quizler 5 sorudan oluşur; ancak ödev 5 oturumda toplam en az 100 farklı soru istemektedir ($5 \times 5 = 25$ soru hedefi karşılamaz). Bu nedenle veri modeli gerçek sistemle aynı tutulmuş, yalnızca oturum başına soru sayısı **20**'ye çıkarılmıştır. Modelde soru sayısı için bir üst sınır yoktur; sayı değişirse yalnızca örnek veri betiği değişir.

Bu yorum ders hocasına e-posta ile danışılmıştır; yanıt geldiğinde bu bölüm güncellenecektir.

2.3 Puanlama kuralı

- **Oturum yüzdesi** = doğru cevap sayısı ÷ oturumdaki soru sayısı × 100.
- Yanlış cevap ve boş soru **0** sayılır; eksi puan yoktur.
- Boş soru için ayrı kayıt tutulmaz: kullanıcının cevap kaydı olmayan soru “boş” kabul edilir.
- Kullanıcının toplam yüzdesi tabloda **saklanmaz**; cevaplardan sorguyla hesaplanır (bkz. 5.4).

3. Teknoloji Seçimi: Neden SQLite?

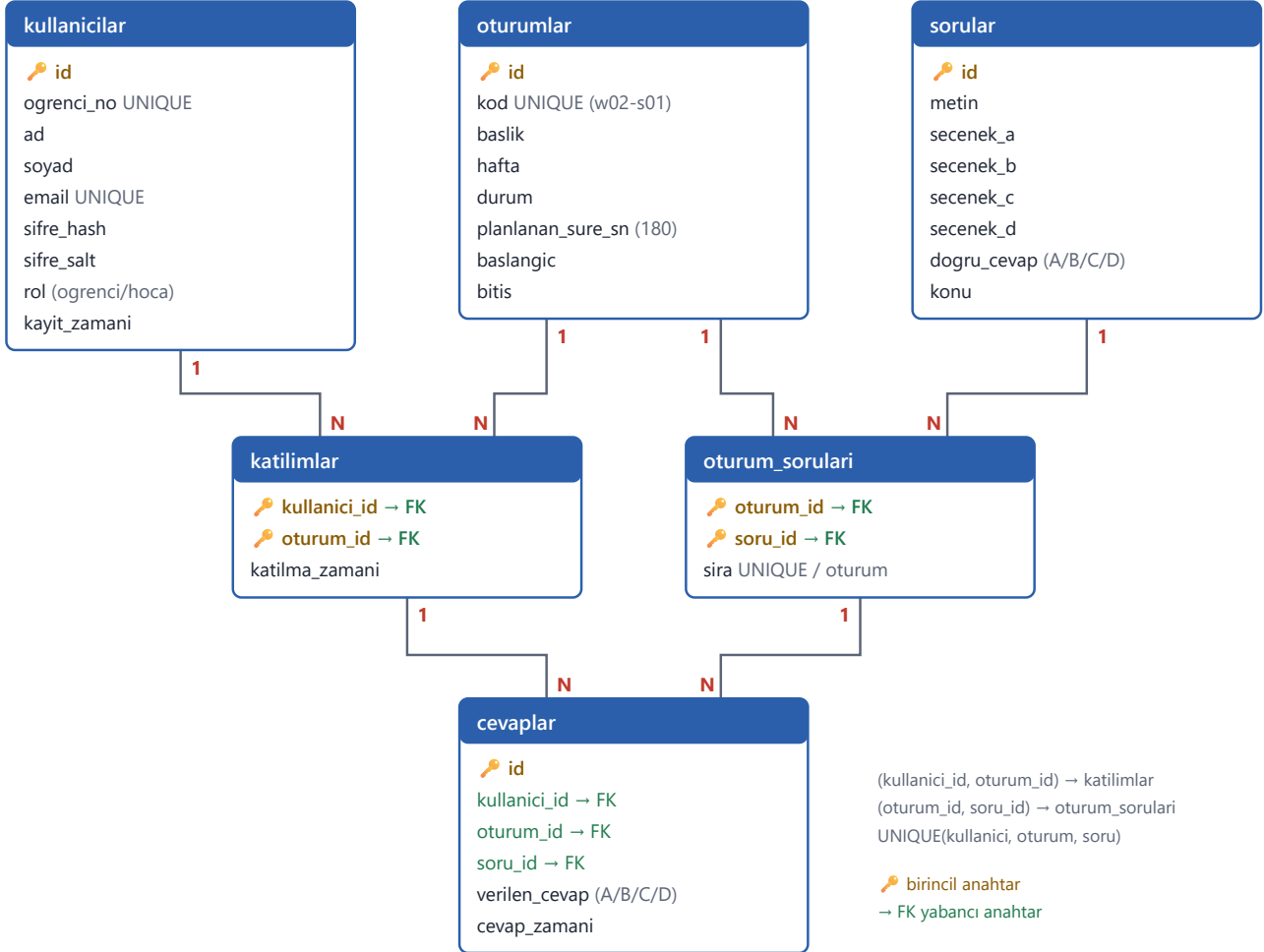
	SQLite	PostgreSQL / MySQL
Mimari	Gömülü (sunucusuz); motor uygulamanın içinde kütüphane olarak çalışır	İstemci-sunucu; ayrı bir servis, port üzerinden bağlantı
Kurulum	Tek bir çalıştırılabilir dosya ya da hiçbir şey (Python'da yerleşik)	Servis, yapılandırma, yönetici hesabı
Verinin yeri	Tek bir <code>.db</code> dosyası	Sunucunun yönettiği veri dizini
Eşzamanlı yazma	Aynı anda tek yazıcı	Çok sayıda eşzamanlı yazıcı
Kullanıcı/yetki	Yok (dosya izinleri)	Ayrıntılı rol ve yetki sistemi

Bir sınıfın quiz sistemi için SQLite yeterlidir; teslimi tek dosya olduğu için kolaydır. Dikkat edilmesi gereken nokta: SQLite'ta yabancı anahtar denetimi varsayılan olarak **kapalıdır**; her bağlantıda `PRAGMA foreign_keys = ON;` çalıştırılmalıdır.

Kullanılan araçlar: SQLite 3.53.4 (komut satırı), Python 3 + yerleşik `sqlite3` modülü, DB Browser for SQLite (görsel inceleme), Git + GitHub (sürüm takibi).

4. Veri Modeli (Adım 1)

4.1 Varlık-İlişki Diyagramı



4.2 Tabloların görevleri

Tablo	Görevi	Önemli kural
kullanıcılar	Öğrenciler ve hoca: ad, soyad, öğrenci no, e-posta, şifre, rol, kayıt zamanı	E-posta ve öğrenci no tekrarlanamaz; şifre düz metin değil, <i>hash</i> + <i>salt</i> olarak saklanır
oturumlar	Her bir quiz (ör. Week 2 · Session 1)	Durum yalnızca <i>baslamadi</i> / <i>suruyor</i> / <i>bitti</i> ; zamanlar durumla tutarlı olmalı
sorular	Soru bankası: metin, 4 şık, doğru cevap, konu	Doğru cevap yalnızca A, B, C veya D
oturum_sorulari	Hangi soru hangi oturumda, kaçınıcı sırada	Aynı soru aynı oturuma iki kez eklenemez; oturum içinde sıra numarası tekildir
katilimler	Öğrencinin quize katıldığı an (henüz cevap vermemiş olsa bile)	Canlı takipte “katıldı ama cevap vermedi” bilgisini sağlar
cevaplar	Öğrencinin bir soruya verdiği cevap ve zamanı	Öğrenci–oturum–soru başına tek kayıt; cevap değişince kayıt güncellenir

4.3 İlişkiler

- oturumlar ↔ sorular: çoka-çok (N:M)**. Bir oturumda çok soru, bir soru (ileride) birden çok oturumda olabilir. Bu ilişki ara tablo `oturum_sorulari` ile çözülür ve `sira` bilgisi bu ara tabloda tutulur.
- kullanıcılar ↔ oturumlar: çoka-çok (N:M)**, ara tablo `katilimler` .
- cevaplar**, bir katılıma (kullanıcı + oturum) ve bir oturum–soru eşleşmesine (oturum + soru) bağlıdır. Böylece cevap üç varlığa birden — kullanıcı, oturum, soru — tutarlı biçimde bağlanır.

5. Tasarım ve Veri Bütünlüğü Kuralları

Temel ilke: **kurallar uygulama kodunda değil, veritabanının kendisinde** uygulanır. Böylece veriye hangi araçla (Python, DB Browser, komut satırı) erişilirse erişilsin hatalı veri girilemez.

5.1 Ödevin özellikle istediği kurallar

Kural	Nasıl sağlanıyor?
Aynı soru aynı oturuma iki kez eklenemez	<code>oturum_sorulari</code> tablosunun birincil anahtarı (<code>oturum_id</code> , <code>soru_id</code>) çiftidir.
Cevap, sorunun gerçekten bulunduğu oturuma ait olmalı	<code>cevaplar(oturum_id, soru_id)</code> çifti, <code>oturum_sorulari</code> tablosuna bileşik yabancı anahtar ile bağlıdır. Oturumda olmayan soruya verilen cevabı veritabanı reddeder.
Tekrar cevap kaydı oluşamaz	<code>UNIQUE(kullanici_id, oturum_id, soru_id)</code> . Cevap değiştirmek yeni satır eklemeyi, mevcut satırı günceller (UPSERT).
Olmayan kullanıcı / soru için cevap verilemez	Yabancı anahtarlar + <code>PRAGMA foreign_keys = ON;</code>

5.2 Ek iş kuralları

- **Katılmadan cevap verilemez:** `cevaplar(kullanici_id, oturum_id) → katilimler` bileşik yabancı anahtarı.
- **Yalnızca süren oturuma cevap verilir:** Oturum `suruyor` değilse cevap ekleme/güncelleme bir *trigger* ile reddedilir. Böylece hoca quizi bitirdikten sonra cevap değiştirilemez.
- **Aynı anda tek oturum sürebilir:** kısmi benzersiz indeks `tek_aktif_oturum ON oturumlar(durum) WHERE durum = 'suruyor'`.
- **Durum geçişleri tek yönlüdür:** `baslamadi → suruyor → bitti`; geriye dönüş trigger ile engellenir.
- **Zaman tutarlılığı (CHECK):** `baslamadi` ise başlangıç ve bitiş boş; `suruyor` ise başlangıç dolu, bitiş boş; `bitti` ise ikisi de dolu ve bitiş $\geq$ başlangıç.
- **Değer kısıtları (CHECK):** şıklar ve doğru cevap yalnızca A/B/C/D; rol yalnızca öğrenci/hoca; planlanan süre pozitif.

5.3 Güvenlik: şifrelerin saklanması

Şifreler düz metin olarak saklanmaz. Her kullanıcı için rastgele bir *salt* üretilir ve şifre, salt ile birlikte tek yönlü bir özet (hash) fonksiyonundan geçirilir (PBKDF2-SHA256). Veritabanı ele geçirilse bile şifrelerin kendisi okunamaz; aynı şifreyi kullanan iki kullanıcının hash'i de farklı olur.

5.4 Normalizasyon: türetilen bilgi saklanmaz

Kullanıcının oturum yüzdesi ve toplam yüzdesi tabloya yazılmaz; her seferinde `cevaplar` ve `sorular` tablolarından sorguyla (VIEW) hesaplanır. Ayrıca saklansaydı her cevap değişikliğinde güncellenmesi gerekir, unutulduğunda iki bilgi çelişirdi. Aynı ilkeyle süre aşımı (+1, +10, +30 sn) da saklanmaz: `(bitis - baslangic) - planlanan_sure_sn` olarak hesaplanır.

5.5 Canlı takip

“Canlı takip” için ayrı bir mekanizma gerekmez: hoca paneli veya sorgu, güncel veriyi belirli aralıklarla yeniden okur. Bir oturumun anlık durumu, katılımcı sayısı, cevaplanan soru sayısı ve geçen süre tek sorguyla izlenebilir.

5.6 Tasarım tercihi: ENUM ve saklama maliyeti

Az sayıda sabit değer alan sütunlar (ör. haftanın günleri) metin yerine küçük kodlarla (ENUM) tutulursa yer kazanılır: `'wednesday'` 9 bayt, 7 değerlik bir kod 3 bit.

- **SQLite'ta ENUM tipi yoktur.** Taklit yolları: `sayı kodu ( durum INTEGER CHECK (durum IN (0,1,2)) )` ya da ayrı tablo + yabancı anahtar.
- **SQLite bit düzeyinde saklamaz;** en küçük birim bayttır. Küçük bir sayı 1 bayt, 0 ve 1 ise neredeyse hiç yer kaplamaz.

Sütun	Şu an	Kod ile	Satır	Kazanç
oturumlar.durum	'baslamadi' 5–9 bayt	1 bayt	5	~30 bayt – önemsiz
kullanici.rol	'ogrenci' 4–7 bayt	0–1 bayt	61	~400 bayt – önemsiz
cevaplar.verilen_cevap	'A' 1 bayt	1 bayt	3018	Yok – zaten en küçük hâli
sorular.dogru_cevap	'A' 1 bayt	1 bayt	100	Yok
Zaman sütunları	'2026-09-25 10:00:00' 19 bayt	4–6 bayt (Unix zamanı)	~3300	~45 KB – tek anlamlı kazanç

- **En kalabalık tablo (cevaplar) zaten verimli:** cevap tek karakter. İsrar yalnızca 5 ve 61 satırlık tablolarda var ve önemsiz.
- **Değiştirmenin bedeli okunabilirlik:** WHERE durum = 'suruyor' yerine WHERE durum = 1 ; DB Browser'da bitti yerine 2 , zaman yerine 1790668800 görünür. Trigger, VIEW, sorgu ve testlerin hepsi değişir.
- **Mevcut yapı enum'un güvencesini zaten sağlıyor:** TEXT + CHECK (durum IN (...)) yalnızca izin verilen değerleri kabul eder (testlerde 'iptal' reddedildi). Sağlamadığı tek şey birkaç KB'lık tasarruf.

Karar: Bu ölçekte okunabilirlik seçildi, yapı değiştirilmedi. Sistem büyürse (milyonlarca cevap) önce zaman damgaları INTEGER'a (Unix zamanı), sonra durum/rol sayı koduna çevrilir.

6. Şema (Adım 2) – `schema.sql`

Çalıştırma: `sqlite3 quiz.db ".read schema.sql"` → 6 tablo, 5 indeks, 4 trigger, 2 görünüm (Bölüm 8.1).

6.1 Kısıt türleri ve kullanıldığı yerler

Kısıt	Ne işe yarar?	Örnek
PRIMARY KEY	Her satırın benzersiz kimliği	<code>oturum_sorulari(oturum_id, soru_id)</code>
FOREIGN KEY	Bağlanılan kayıt gerçekten var olmalı	<code>cevaplar(oturum_id, soru_id) → oturum_sorulari</code>
UNIQUE	Tekrarı engeller	<code>email, ogrenci_no, (kullanici, oturum, soru)</code>
NOT NULL	Boş bırakılamaz	ad, soyad, soru metni, şıklar
CHECK	Değer kuralına uymalı	<code>cevap ∈ {A,B,C,D}</code> ; durum–zaman tutarlılığı
DEFAULT	Değer verilmezse otomatik doldurulur	<code>kayit_zamani, durum='baslamadi'</code> , süre 180 sn

6.2 Trigger'lar

CHECK yalnızca aynı satıra bakabilir. Başka tabloya bakan (oturumun durumu) veya eski/yeni değeri karşılaştıran kurallar trigger ile yazıldı.

Trigger	Engellediği durum
<code>trg_oturum_durum_gecisi</code>	Durumun geri gitmesi (ör. bitti → suruyor)
<code>trg_katirim_bitmis_oturum</code>	Bitmiş oturuma katılım
<code>trg_cevap_ekle_aktif_oturum</code>	Süren oturum dışında cevap ekleme
<code>trg_cevap_guncelle_aktif_oturum</code>	Oturum bittikten sonra cevap değiştirme

6.3 İndeksler

PRIMARY KEY ve UNIQUE sütunları otomatik indekslenir. Ek olarak: katılımcıları oturuma göre, cevapları oturum+soruya göre, oturumları duruma göre ve soruların hangi oturumlarda olduğunu bulan 4 indeks eklendi. Ayrıca `tek_aktif_oturum` **kısmi benzersiz indeks** bir kural görevi görür: UNIQUE yalnızca `durum = 'suruyor'` olan satırlara uygulanır, böylece aynı anda ikinci bir quiz başlatılamaz.

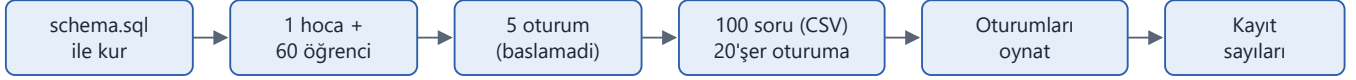
6.4 Hızlı doğrulama

Deneme	Sonuç
Başlamamış oturuma cevap	Reddedildi (trigger)
Oturumda olmayan soruya cevap	Reddedildi (FOREIGN KEY)
Aynı soruya ikinci cevap satırı	Reddedildi (UNIQUE) – UPSERT ile güncelleme kabul
Olmayan kullanıcı ile cevap	Reddedildi (FOREIGN KEY)
Bitiş zamanı olmadan "bitti"	Reddedildi (CHECK)
bitti → suruyor geri dönüş	Reddedildi (trigger)
Bitmiş oturumda cevap değiştirme	Reddedildi (trigger)
Bir quiz sürerken ikincisini başlatma	Reddedildi (kısmi UNIQUE indeks)

7. Örnek Veri (Adım 3) – seed.py

Çalıştırma: `python seed.py` → `quiz.db` silinir, `schema.sql` ile yeniden kurulur ve veri eklenir (~2 sn). Sabit rastgelelik tohumu (`random.seed(42)`) sayesinde her çalıştırmada aynı veri oluşur.

7.1 Akış



7.2 Üretilen veri

Tablo	Kayıt	Not
kullanıcılar	61	60 öğrenci + 1 hoca; e-posta = öğrenci no (20240001@ogrenci.edu.tr)
oturumlar	5	3 bitti, 1 sürüyor, 1 başlamadı
sorular	100	SQL konulu, sorular.csv dosyasından; doğru şıklar A/B/C/D arasında eşit dağıtıldı
oturum_sorulari	100	Her oturuma 20 soru, sıra 1–20
katilimler	206	Öğrencilerin ~%90'ı katılır
cevaplar	3018	Soruların ~%8'i boş bırakılır; bazı cevaplar sonradan değiştirilir (UPSERT)

7.3 Önemli noktalar

- Gerçek akış izlenir:** oturum `baslamadi` olarak eklenir → başlatılır → cevaplar girilir → bitirilir. Bitmiş oturuma doğrudan cevap eklenemediği (trigger) için başka yol yoktur.
- Gerçekçi sonuçlar:** her öğrencinin %40–95 arası bir başarı oranı vardır; doğru/yanlış cevap buna göre seçilir.
- Süre:** bitmiş 3 oturum tam 180 sn (3 dk) sürer. Uzatma, `uzatmalar` listesiyle eklenebilir; şimdilik 0.
- Canlı oturum:** 4. oturum, betik çalıştığı andan 30 sn önce başlamış gibi kurulur; öğrenciler 0–5 soruya ulaşmıştır (normal tempo: soru başına ~9 sn).
- Şifreler:** `hashlib.pbkdf2_hmac` + her kullanıcıya özel 16 baytlık rastgele salt. Deneme şifreleri: öğrenci `ogrenci123`, hoca `hoca123`.
- SQL enjeksiyonuna karşı:** tüm değerler `?` yer tutucularıyla gönderilir, metin birleştirme ile sorgu kurulmaz.

8. Sorgular (Adım 4) – `queries.sql`

Çalıştırma: DB Browser → *Execute SQL*, ya da `sqlite3 quiz.db ".read queries.sql"`. A–C bölümleri yalnızca okur; D bölümü veriyi değiştirir (geri almak için `python seed.py`).

8.1 Görünümler (VIEW) – `schema.sql` içinde

Görünüm	Hesapladığı
<code>v_oturum_durumu</code>	Her oturumun geçen süresi ve 180 sn'yi aşan uzatma süresi (<code>strftime('%s', ...)</code> ile tam saniye)
<code>v_sonucilar</code>	Her katılımcının her oturumdaki doğru, yanlış, boş sayısı ve yüzdesi. Puanlama kuralı yalnızca burada yazılıdır.

Puanlama: doğru = 1, yanlış = 0, boş = 0 → yüzde = doğru ÷ oturumdaki soru sayısı × 100. Genel sıralamada payda bitmiş oturumların toplam sorusudur; katılmayan quiz 0 sayılır.

8.2 Sorgu listesi

No	Sorgu	Ödevdeki karşılığı
A1–A3	Tablo kayıt sayıları, farklı soru sayısı, oturum başına soru sayısı	Kayıt sayılarının doğrulanması
B1	Tüm oturumların durumu, geçen süre, uzatma	Oturumun anlık durumu ve cevaplanma ilerlemesi (canlı takip)
B2	Süren oturum: katılımcı, katılmayan, verilen cevap, tamamlanma %	
B3	Süren oturum: soru bazında cevaplayan sayısı	
B4	Süren oturum: öğrenci bazında ilerleme (ör. 14/20)	
C1	Bir oturumun sonuçları: doğru / yanlış / boş, "18/20 · %90"	Oturum sonuçları
C2	Oturum özetleri: katılımcı, ortalama, en yüksek, en düşük	
C3	Genel yüzdeye göre ilk 10 öğrenci	En yüksek puanlı kullanıcılar
C4	Doğru oranı en düşük 5 soru	Ek
D0–D7	Süreni bitir → yenisini başlat → katıl → cevapla → cevabı değiştir → bitir → sonuç	Cevap kaydetme ve sonuç üretme akışı

8.3 Örnek çıktılar

kod	katılımcı	katılmayan	verilen cevap	beklenen	tamamlanma
w02-s01 (B2)	49	11	137	980	%14.0

kod	katılımcı	ortalama	en yüksek	en düşük
w01-s01 (C2)	51	%60.2	%100	%20
w01-s02	53	%63.6	%100	%25
w01-s03	53	%65.1	%95	%35

Not: Süren oturum, `seed.py` çalıştırılmadan 30 sn önce başlamış kabul edilir; kimse bitirmezse süre ve uzatma artmaya devam eder. `python app.py` açılışta 5 dakikadan eskiyse veriyi yeniler (Bölüm 10.2).

D akışının sonucu: öğrenci önce B, sonra A dedi → `cevaplar` tablosunda **tek** satır, değeri A; oturum bitince sonuç 1 doğru, 19 boş, %5.

9. Veri Bütünlüğü Testleri (Adım 5) – testler.py

Çalıştırma: `python testler.py` . Her test kurallara aykırı bir işlem dener ve reddedildiğini doğrular; birkaç test de doğru işlemlerin kabul edildiğini doğrular. Sonunda `rollback` ile her şey geri alınır, `quiz.db` değişmez.

Sonuç: 29 kontrolün 29'u beklendiği gibi.

Denenen işlem	Sonuç	Engelleyen
Ödevin açıkça istedikleri		
Olmayan kullanıcı adına cevap	Reddedildi	FOREIGN KEY
Olmayan soruya cevap	Reddedildi	FOREIGN KEY
Oturumda bulunmayan soruya cevap	Reddedildi	Bileşik FK
Aynı soruya ikinci cevap satırı (tekrar kayıt)	Reddedildi	UNIQUE
Aynı soruya cevap değiştirme (UPSERT)	Kabul – yine tek satır	—
Ek kurallar		
Katılmamış kullanıcının cevabı	Reddedildi	Bileşik FK
Bitmiş / başlamamış oturuma cevap, bitmiş oturumda cevap değiştirme	Reddedildi	Trigger
Bitmiş oturuma katılım, aynı oturuma ikinci katılım	Reddedildi	Trigger / PK
Aynı soru ya da aynı sıra no bir oturumda iki kez	Reddedildi	PK / UNIQUE
Cevapları olan soruyu silme	Reddedildi	FOREIGN KEY
İkinci oturumu başlatma, durumu geri alma	Reddedildi	Kısmi UNIQUE / Trigger
Bitişsiz "bitti", bitiş başlangıçtan önce, geçersiz durum	Reddedildi	CHECK
Tekrarlanan e-posta (büyük harfle bile), öğrenci no, oturum kodu	Reddedildi	UNIQUE (NOCASE)
Numarasız öğrenci, geçersiz e-posta, geçersiz şık	Reddedildi	CHECK

10. Arayüz (Adım 7) – app.py

Çalıştırma: `pip install -r requirements.txt` (bir kez), sonra `python app.py` → `http://127.0.0.1:5000`.
İlke: **küçük ama ödevin her maddesini gösteren** bir arayüz.

10.1 Ödev maddesi → ekran

Ödevin istediği	Arayüzde	SQL
50+ kullanıcı, 5 oturum, 100 soru	Hoca panelinde sayaçlar	A1
Başlamamış / süren / bitmiş oturum ve zamanlar	Oturum tablosu, Başlat / Bitir , süre ve uzatma (+sn)	B1, D0/D1/D6
Canlı takip: anlık durum ve ilerleme	Hoca <i>Başlat</i> deyince quiz öğrencinin ekranında belirir, <i>Bitir</i> deyince kaybolur (≤ 1 sn). Hoca paneli yeni katılım/cevapta kendini yeniler. Süre sayacı saniye saniye işler.	B2, B4
Cevap kaydetme akışı	Öğrenci: Katıl → şıkka tıkla → değiştir	D2, D3/D4
Oturum sonuçları	Sonuç sayfası, öğrencinin “Sonuçlarım” listesi (18/20 · %90)	C1, C2
En yüksek puanlı kullanıcılar	Hoca panelinde ilk 10	C3
Veri bütünlüğü	Reddedilen işlem sarı “veri bütünlüğü kuralı” kutusunda: Türkçe açıklama + veritabanı mesajı	Kısıt / trigger

10.2 Önemli noktalar

- **Kurallar veritabanında:** arayüzde ayrıca kontrol yazılmadı. Ör. hoca bir quiz sürerken ikincisini başlatmak isterse ya da öğrenci bitmiş quizde cevap değiştirmek isterse ekranda sarı kural kutusu çıkar (ör. “Aynı anda yalnızca bir quiz sürebilir”) ve altında veritabanının asıl mesajı görünür.
- **Giriş:** girilen şifre kayıtlı salt ile hashlenir ve kayıtlı hash ile karşılaştırılır; şifrenin kendisi hiçbir yerde saklanmaz.
- **Canlı (live):** sayfalar saniyede bir `/canlı` adresine küçük bir soru sorar (cevap ör. `{"oturum_id": 4, "imza": "4-49-137"}`; imza = süren quiz + katılımcı + cevap sayısı). Değer değiştiyse sayfa yenilenir, değişmediyse hiçbir şey olmaz. Her saniye bütün sayfayı yüklemek yerine birkaç baytlık bir kontrol yapılır.
- **Süre sayacı:** tarayıcıda saniye saniye işler, 180 sn aşılmıca uzatmayı gösterir (3:12 (+12 sn)).
- **Açılışta bayat veri kontrolü:** örnek verideki süren quiz `seed.py` anından beri işler. Hoca projeyi günler sonra açtığına “günlerdir sürüyor” görünmesin diye `python app.py` açılışta bir kez bakar: süren quiz 5 dakikadan (`BAYAT_ESIGI_SN = 300`) uzun süredir açıksa `seed.py` çalıştırılır. Uygulama çalışırken bir daha bakılmaz; 3 dakikalık eşik seçilmedi çünkü gerçek uzatmaları da bayat sayardı.
- **Kendini anlatan arayüz:** deneme hesapları ve “iki ayrı adres” notu giriş sayfasında; README başında “Hızlı inceleme”.
- **Yetki:** öğrenci hoca sayfalarını açamaz (girişe yönlendirilir).
- **Yapı:** `app.py` (~250 satır) + 5 şablon (`base` , `giris` , `ogrenci` , `hoca` , `sonuc`). Sorgular `queries.sql` ile ayrıdır.

11. Karar Özeti

Proje boyunca alınan kararlar ve gerekçeleri, tek yerde.

11.1 Veri modeli

Karar	Neden
Oturum = tek quiz (Session)	Başlık, durum, başlangıç/bitiş tek quize karşılık gelir. Hocaya danışıldı, yanıt bekleniyor.
Oturum başına 20 soru	Gerçek quizler 5 soru; ödev 5 oturumda 100 farklı soru istiyor. Modelde sınır yok.
Kurallar veritabanında	Hangi araçla erişilirse erişilsin hatalı veri girilemez; arayüzde tekrar yazılmadı.
Cevap değiştirme = UPSERT, bitince trigger kilidi	Gerçek sistemde quiz açıkken değiştirme serbest; öğrenci-soru başına tek satır.
Aynı anda tek quiz (kısmi UNIQUE)	Test sırasında fark edilen eksik kapatıldı.
Puan ve süre VIEW ile hesaplanır	Türetilen bilgi saklanmaz (normalizasyon).
Şifre: PBKDF2-SHA256 + salt	Veritabanı ele geçse bile şifreler okunamaz.
ENUM yerine TEXT + CHECK	Bu ölçekte kazanç birkaç KB; okunabilirlik seçildi (Bölüm 5.6).

11.2 Örnek veri

Karar	Neden
3 bitti + 1 sürüyor + 1 başlamadı	Üç durum veride hazır görünür, canlı takip ilk açılışta dolu. "Süren quiz olmasın" denendi: canlı takip boş kaldı, geri dönüldü.
Bitmiş quizler tam 3 dk	Sade veri; uzatma <code>uzatmalar</code> listesiyle eklenebilir.
Süren quiz seed anından 30 sn önce başlamış	İki sekmeyi açmaya 3 dk dolmadan vakit kalsın. İlerleme buna uygun: 0–5 soru.
<code>random.seed(42)</code> , sorular CSV'de, ? yer tutucuları	Yeniden kurulabilir veri · veri/kod ayrımı · SQL enjeksiyonuna karşı koruma.

11.3 Arayüz

Karar	Neden
Flask	Python'un en sade web kütüphanesi; proje zaten Python, tek ek paket.
Küçük ama ödevin her maddesini gösteren 4 ekran	Ödev "küçük ve çalışan" arayüz istiyor (Bölüm 10.1).
Live: saniyede bir /canlı kontrolü	Tam sayfa yenileme (5 sn) yerine birkaç baytlık kontrol; değişince yenilenir.
Açılışta bayat veri kontrolü, eşik 5 dk	Günler sonra açılınca "günlerdir sürüyor" görünmesin. Sayfa açılışında seed live akışı bozardı; 3 dk gerçek uzatmaları da bayat sayardı.
Hoca 127.0.0.1 , öğrenci localhost	Giriş çerezde tutulur; aynı adresteki sekmeler birbirini kapatır. İki adres ayrı site sayılır (alternatif: farklı tarayıcı / gizli pencere).
Sarı "veri bütünlüğü kuralı" kutuları, giriş sayfasında deneme hesapları	Proje tek başına incelenecek: kural ihlali hata gibi görünmesin, giriş için README gerekmesin.
Canlı simülasyon yapılmadı	Seed'deki süren quiz canlı takibi zaten dolu gösteriyor.